

Module 16: Integrating AI Models into Full Stack Applications

Module Overview

- Full-stack AI solutioning concludes with integrating machine learning models into software applications for real-world business impact.
- Model persistence involves storing trained parameters and data securely for long-term accessibility and reuse in production environments.
- Deployment of models enables scoring of new data through RESTful APIs and orchestration tools like MLflow for automation.
- Integration with frontend applications ensures seamless data flow between user inputs and model predictions for decision-making.

Model persistence: Store the trained model, including parameters and data, for long-term use

Deployment: Make the model available in production to analyse new, unseen data

Integration: Connect the model to applications for seamless input processing and automated decision-making

- Business value captured through automation leads to improved efficiency, cost savings, and potential revenue growth from AI-driven processes.

Overview of Model Persistence and Serving

- Machine learning model development involves data preprocessing, feature engineering, algorithm selection, and performance evaluation across various combinations.

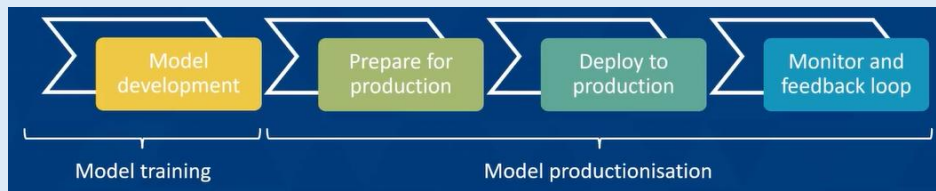
Data is collected and preprocessed through cleaning, transformation, and reduction.

Feature engineering balances complexity, bias, and variance.

Models are built using various features, algorithms, and hyperparameters.

Models are evaluated using performance metric to determine the most effective one.

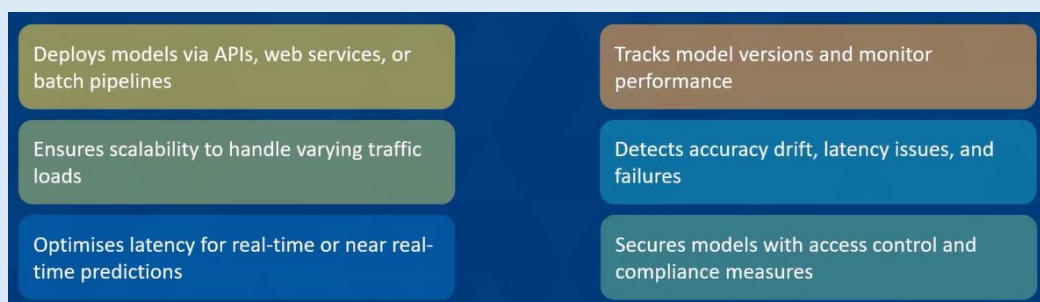
- Validation through training and testing partitions is essential to detect overfitting and ensure generalisability to unseen data.
- Model productionisation follows model development and forms part of the broader machine learning operations (MLOps) lifecycle.



- Model persistence enables saving and reusing trained models efficiently, supporting reproducibility, version control, and deployment.

- Stores a trained machine learning model, enabling its reuse without the need for retraining
- Advantages:
 - Saves time and computational resources
 - Ensures consistent model use across sessions and applications
 - Enables easy deployment into real-world systems and APIs
 - Supports version control and model tracking

- Model serving facilitates real-time or batch prediction by deploying models through scalable, monitored, and secure systems.



Saving and Loading scikit-learn Models

- Scikit-learn supports both supervised and unsupervised learning and is built on top of NumPy, SciPy, and Matplotlib.
- Joblib and Pickle are two common methods for saving and loading trained scikit-learn models for future use.

- Supervised learning: Supports classification and regression
- Unsupervised learning: Supports clustering like k-means, DBSCAN
- Model persistence: Saves and reloads models without retraining
- Persistence methods:
 - joblib: Efficient for large models
 - pickle: General-purpose, less efficient

- Joblib is optimised for large NumPy arrays, offering faster performance and better memory efficiency for big models.

```
# Save the trained model to a file
joblib.dump(model, 'random_forest_model.pkl')

# Load the trained model
loaded_model = joblib.load('random_forest_model.pkl')

# Make predictions
predictions = loaded_model.predict(X_test)
```

Saves models faster and uses less storage

Handles large models effectively

Easy to use and implement

- Pickle is more general-purpose and compatible with all Python objects but is slower and less efficient for large data.

- pickle is a built-in Python library for serialising and deserialising objects.
- It allows saving Python objects, including those created from classes, to a file for persistence.
- Unlike joblib, pickle is not optimised for handling large numerical data efficiently.
- Advantages of pickle:
 - Works with all Python objects, not just NumPy arrays.
 - More general-purpose compared to joblib.
- Disadvantages of pickle:
 - Not optimised for large NumPy arrays.
 - Slower and consumes more memory than joblib.

```
# Save model using pickle
with open('random_forest_model.pkl', 'wb') as file:
    pickle.dump(model, file)

# Load model using pickle
with open('random_forest_model.pkl', 'rb') as file:
    loaded_model = pickle.load(file)

# Make predictions
predictions = loaded_model.predict(X_test)
```

Activate Windows
Go to Settings to activate Windows.

- A decision tree classifier trained on the Iris dataset can be saved, loaded, and used to predict flower species accurately.

```
# Access tree parameters via the "tree_" attribute
n_nodes = loaded_model.tree_.node_count # Total number of nodes
n_features = loaded_model.n_features_in_ # Number of input features
thresholds = loaded_model.tree_.threshold # Threshold values for each split
features = loaded_model.tree_.feature # Feature indices used for each split
print(n_nodes)
print(n_features)
print(thresholds)
print(features)
```

```
✓ 0.0s
5
4
[ 2.44999999 -2.      1.75   -2.      -2.      ]
[ 2 -2  3 -2 -2]
```

Activate Windows
Go to Settings to activate Windows.

Saving and Loading Keras Models

- Keras supports multiple model persistence methods including saving the entire model, saving weights only, and saving architecture separately.
- Saving the entire Keras model ensures seamless reusability, including architecture, weights, and training configurations for inference and further training.

Saving the full model ensures easy reloading with all components included.

```
# Save the model
model.save('my_model')

# Load the model
new_model = keras.models.load_model('my_model')

# Verify that the loaded model works
new_model.summary()
```

- Saving only model weights is useful in scenarios such as transfer learning, architecture modification, or deploying models with updated input dimensions.

Save only the trained weights, not the full model

Useful for reinitialising models with new architectures

Add new layers for transfer learning while retaining pretrained weights

Change input shape without retraining earlier layers

Modify activation functions or layer properties efficiently

Requires recreating and compiling the model before loading weights

```
# Recreate the model architecture
new_model = keras.Sequential([
    keras.layers.Dense(64, activation='relu', input_shape=(10,)),
    keras.layers.Dense(1, activation='sigmoid')
])

# Compile the model (compilation is necessary before loading weights)
new_model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])

# Load the weights
new_model.load_weights('model_weights')

# Verify that the model has the correct weights
new_model.summary()
```

Activate Windows
Go to Settings to activate Windows.

- Sharing just the model architecture, typically in JSON format, allows collaborative development without exposing proprietary or sensitive training data.

```
# Convert model to JSON format
model_json = model.to_json()

# Save JSON to file
with open("model_architecture.json", "w") as json_file:
    json_file.write(model_json)

# Load JSON model architecture
with open("model_architecture.json", "r") as json_file:
    loaded_json = json_file.read()

# Recreate model from JSON
from tensorflow.keras.models import model_from_json
new_model = model_from_json(loaded_json)

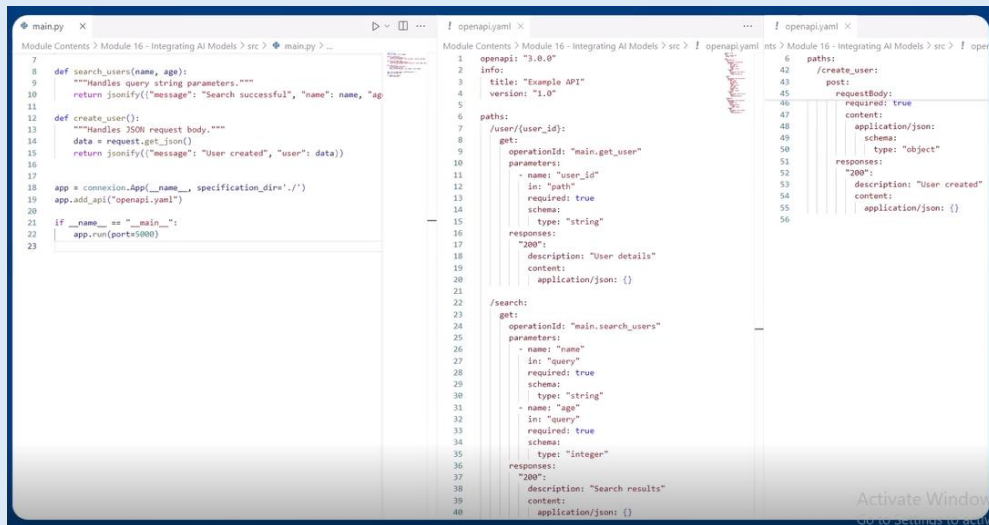
# Compile the model
new_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

- Among the three formats—SavedModel, HDF5, and Keras—the Keras format offers faster load times and smaller file size, making it the most efficient choice.

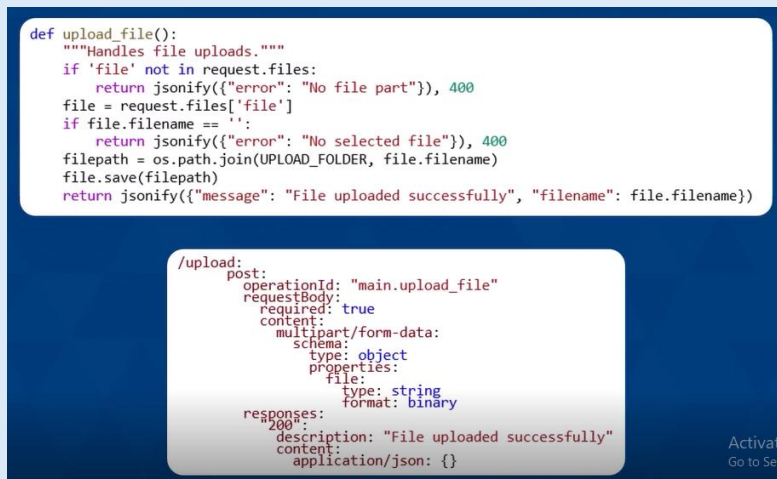
Serving Models via RESTful API Endpoints Using Python with Flask and Connexion

- RESTful API endpoints provide a flexible solution for integrating machine learning models into software applications and business workflows.
- Python web frameworks such as Flask and Connexion support model deployment through structured, customisable API endpoints.

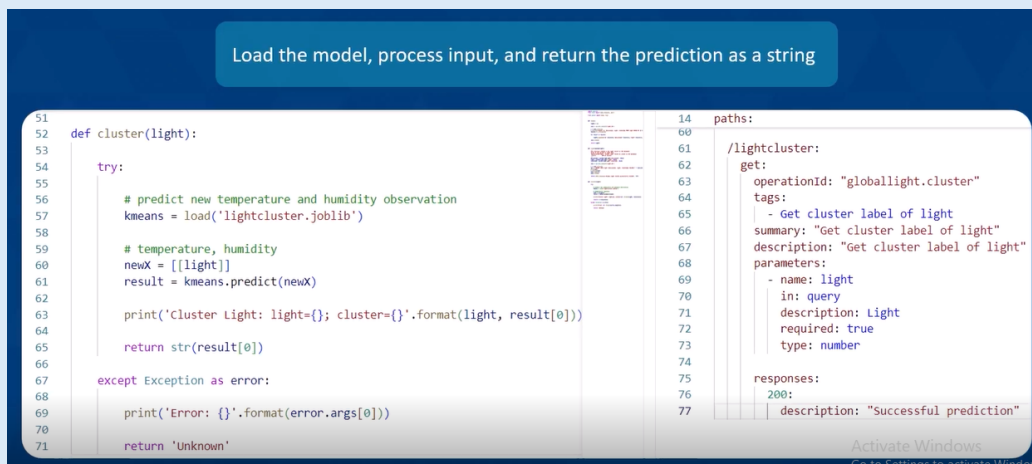
- Model input data can be passed as path parameters, query strings, or JSON payloads depending on the number of variables.



- File upload functionality enables deployment of models that require image or document input through POST request endpoints.



- Vendor-specific tools like TensorFlow Serving and TorchServe offer convenience but reduce flexibility and increase switching costs.



Consuming RESTful API Endpoints from a React Web Application

- Client-side rendering frameworks like React can integrate machine learning model predictions served through RESTful API endpoints.
- Dynamic content and personalised user experiences can be driven by predictions from backend models accessed via HTTP requests.
- The native JavaScript `fetch()` function provides a flexible, promise-based approach to making API calls from React components.

- It is a built-in JavaScript function for HTTP requests.
- It replaces `XMLHttpRequest` for cleaner, flexible syntax.
- It is Promise-based, simplifying asynchronous operations.
- It supports `async/await` and `.then()` chaining.

```
fetch(url)
  .then(response => response.json()) // Convert response to JSON
  .then(data => console.log(data))   // Handle the data
  .catch(error => console.error("Error:", error)); // Handle errors
```

- Axios simplifies API requests by offering built-in error handling and automatic rejection for non-2xx HTTP status responses.

A third-party library designed to simplify API calls and handle errors effectively

```
const AxiosExample = () => {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);

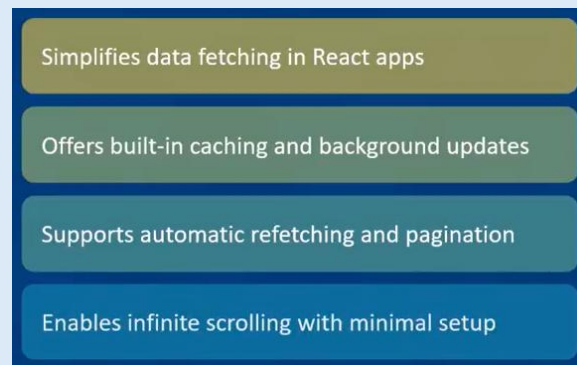
  useEffect(() => {
    axios.get(API_URL)
      .then(response => {
        setData(response.data);
        setLoading(false);
      })
      .catch(error => {
        console.error("Error fetching data:", error);
        setLoading(false);
      });
  }, []);
}
```

- Axios: Automatically rejects promises for non-2xx status codes (e.g., 404, 500)
- `fetch()`: Requires manual checking of `response.ok` and `response.status` for error handling

```
// Using fetch() function
fetch(API_URL) // Non-existent post
  .then(response => {
    if (!response.ok) { // Manually check response status
      throw new Error(`HTTP error! Status: ${response.status}`);
    }
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.error("Fetch error:", error));

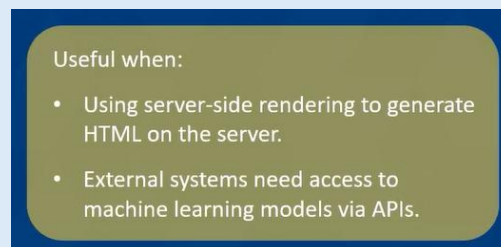
// Using Axios
axios.get(API_URL)
  .then(response => console.log(response.data))
  .catch(error => console.error("Axios error:", error));
```

- React Query enhances data fetching with features such as caching, pagination, and background updates for improved performance.



Consuming RESTful API Endpoints from an Express.js Backend

- Backend applications may require machine learning predictions when using server-side rendering or interacting with external software systems.



- API endpoints can be consumed in Express.js using built-in Fetch API, Axios, or the node-fetch library for older Node.js versions.
- The Fetch API in Node.js version 18 and above operates similarly to browser-based JavaScript environments with async/await support.
- Axios provides a consistent interface across environments and offers enhanced error handling compared to the Fetch API.

- The built-in Fetch API in Node.js (version 18 and onwards) allows API invocation using async/await for asynchronous handling.
- Axios, a third-party library, works similarly to fetch() in the Node.js environment and offers additional benefits.
- For older Node.js versions, the node-fetch library can be used, with the fetch() function requiring installation and import from the library.

- Backend API calls can be optimised using in-memory caching solutions like node-cache or Redis to reduce redundant requests.

- API calls in Express.js can be optimised with caching to prevent excessive calls.
- The node-cache library provides in-memory caching for Node.js applications.
- Redis, an open-source in-memory data store, can be used as a cache, database, or message broker.

```
import NodeCache from 'node-cache';
const cache = new NodeCache({ stdTTL: 60 });

app.get('/cached-data', async (req, res) => {
  const cachedData = cache.get('posts');
  if (cachedData) return res.json(cachedData);

  try {
    const response = await axios.get(API_URL);
    cache.set('posts', response.data);
    res.json(response.data);
  } catch (error) {
    res.status(500).json({ error: 'Error fetching data' });
  }
});
```

Consuming RESTful API Endpoints from a Flask Backend

- Backend systems built with Flask and Connexion must rely on API endpoints to interact with external machine learning models.
- Connexion provides structured API endpoint definitions using OpenAPI specifications, with built-in validation for improved reliability.
- The requests library in Python is commonly used for consuming external APIs by handling GET, PUT, POST, and DELETE operations.

- Install it using `pip install requests`
- Define API endpoints using HTTP methods
- Use `requests.get(url, params={"id": 123})` to pass query parameters
- Use `.json()` to parse responses and `.status_code` to check status

```
import requests

response = requests.get(API_URL)

# Print response data
if response.status_code == 200:
    print("GET Response:", response.json()) # JSON response
else:
    print("Error:", response.status_code)
```

- JSON-formatted data is easily managed in Python using dictionaries, allowing seamless integration with PUT and POST request bodies.

`requests.put(url, json=data)` sends JSON data to a PUT endpoint

HTTP status code 201 indicates successful data creation

JSON in Python is similar to a dictionary, making it easy to handle

``data=data`` sends the information in a form-encoded format

```
import requests

data = {
  "data1": "value1",
  "data2": "value2",
  "data3": "value3"
}

response = requests.put(API_URL, json=data) # Sending JSON data with a dict

if response.status_code == 201:
    print("PUT Response:", response.json())
else:
    print("Error:", response.status_code)
```

Activate Windows
Go to Settings to activate Windows

requests.post(url, json=data) sends an update request to a POST endpoint

HTTP status code 200 indicates a successful data update

```
import requests

data = {
    "data1": "value1",
    "data2": "value2",
    "data3": "value3"
}

response = requests.post(API_URL, json=data) # Sending JSON data with a dict

if response.status_code == 200:
    print("POST Response:", response.json())
else:
    print("Error:", response.status_code)
```

- HTTP status codes such as 200, 201, and 204 are useful for confirming the success of data retrieval, creation, updating, or deletion.

To remove a specific record, you can send a DELETE request using `requests.delete(url)`.

The record identifier is usually required for deletion.

Usually DELETE endpoints respond with an HTTP status of 200 OK or 204 No Content.

Identifiers can be passed as named arguments, e.g., delete(url, id=123).

```
import requests

response = requests.delete(API_URL)

if response.status_code == 200:
    print("DELETE Response:", response.json()) # Response body (empty in most cases)
else:
    print("Error:", response.status_code)
```

Serving a Model Using MLflow

- MLflow supports end-to-end management of the machine learning lifecycle, including experiment tracking, model versioning, and deployment.
- MLflow Tracking logs experiment parameters, metrics, and artefacts, ensuring reproducibility and structured experimentation across multiple runs.

MLflow logs models with full details and stores them centrally for easy versioning.

```
import mlflow

mlflow.set_experiment("My_ML_Experiment")

with mlflow.start_run():
    mlflow.log_param("learning_rate", 0.01)
    mlflow.log_metric("accuracy", 0.92)
    mlflow.sklearn.log_model(trained_model, "model")

mlflow.register_model("runs:/<run_id>/model", "MyModel")
```

- Model Registry in MLflow enables version control, governance, and promotion workflows for transitioning models from staging to production.

MLflow manages model versions, approvals, and tracks changes with metadata.

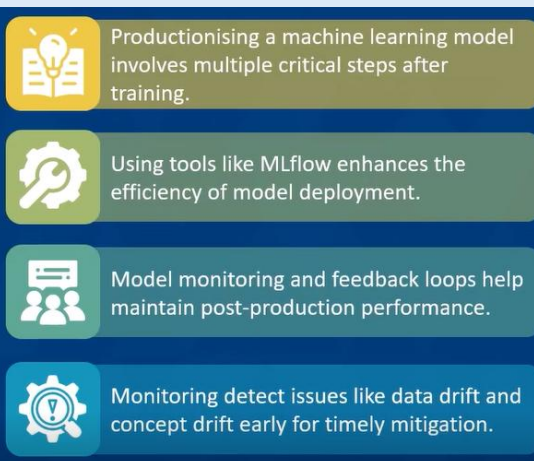
```
from mlflow.tracking import MlflowClient

client = MlflowClient()
client.transition_model_version_stage(
    name="MyModel",
    version=1,
    stage="Production"
)
```

- MLflow supports diverse deployment strategies, including local servers, REST APIs, cloud platforms, and Kubernetes using the ML model format.
- Integration with CI/CD tools and automated pipelines ensures scalable, reliable, and auditable deployment of machine learning models across environments.

Module Summary

- Model lifecycle management extends beyond training, requiring careful productionisation to ensure real-world usability and business impact.
- MLflow platform enhances model deployment efficiency through standardised tracking, reproducibility, and automation of production workflows.
- Post-deployment monitoring helps detect issues like data drift and concept drift, maintaining model accuracy and relevance over time.
- Feedback loops and retraining strategies play a critical role in adapting models to evolving data and operational conditions.



- Continuous learning and exploration of lifecycle tools and practices support long-term success in AI solutioning and model reliability.

THE END